

C++ PROGRAMMING PROJECT

Student Grade Management System

Using Arrays, Functions & Pointers

Course Code	CS 101 – C++ Programming
Level	100 (First Year, Second Semester)
Weight	30% of Final Grade
Submission	29 th May 2026

Academic Year 2026

1. Introduction & Project Overview

This project tests your ability to apply the C++ programming concepts covered during the semester. You will design and build a Student Grade Management System — a console-based application that stores, processes, and reports student academic records.

The system must demonstrate competence in three core areas:

- Arrays — storing and manipulating collections of student data
- Functions — decomposing the program into reusable, well-named routines
- Pointers — using memory addresses to access and modify data efficiently

Learning Objectives

By completing this project you will: (1) apply arrays to manage structured data sets, (2) design modular programs using functions with appropriate parameters and return types, (3)

use pointers and pass-by-reference to manipulate data in memory, and (4) practise problem-solving with real-world scenarios.

2. Problem Description

You are to write a C++ program that manages the grades of students in a class of up to 30 students. The program shall read student names and scores, compute results, and display a formatted report.

2.1 Data to Manage

Your program must keep track of the following information for each student:

Field	Data Type	Description
Student Name	string	Full name of the student (max 50 characters)
Student ID	int	Unique numeric ID (e.g. 20240001)
Score 1 (Quiz)	float	Score out of 20 marks
Score 2 (Mid-Sem)	float	Score out of 30 marks
Score 3 (Exam)	float	Score out of 50 marks
Total Score	float	Sum of all three scores (out of 100)
Grade	char	Letter grade: A, B, C, D, or F
Remark	string	Pass or Fail

2.2 Grading Scale

Total Score (%)	Grade	Remark
80 – 100	A	Excellent — Pass
70 – 79	B	Good — Pass
60 – 69	C	Average — Pass
50 – 59	D	Below Average — Pass
0 – 49	F	Fail

3. Required Program Features

Your program must implement ALL of the following features to obtain full marks.

3.1 Input Module

- Accept input for up to 30 students from the keyboard.
- Validate that each score falls within its allowed range (e.g. Quiz score must be 0–20). Prompt the user to re-enter invalid values.
- Allow the user to enter fewer than 30 students and stop input when they choose.

3.2 Processing Module

- Compute the total score for every student.
- Assign the correct letter grade and remark based on the grading scale in Section 2.2.
- Find the highest and lowest total scores in the class.
- Calculate the class average score.
- Count the number of students who passed and those who failed.

3.3 Output / Report Module

- Display a neatly formatted class report showing all student records in a table.
- Display class statistics: highest score, lowest score, class average, pass count, and fail count.
- Allow the user to search for a student by ID and display their individual result.

3.4 Menu System

The program must present the user with a simple numbered menu:

1. Enter Student Records
2. Compute and Assign Grades
3. Display Full Class Report
4. Display Class Statistics
5. Search Student by ID
6. Exit

Important Note on Arrays

All student data **MUST** be stored in parallel arrays (or an array of structs). Dynamic memory allocation (new/delete) is optional but earns bonus marks. You are **NOT** permitted to use the C++ Standard Template Library (STL) containers such as vector, list, or map.

4. Technical Requirements

4.1 Mandatory Use of Functions

Your solution must be broken into functions. A minimum of six (6) distinct functions are required beyond `main()`. Below are the suggested function signatures — you may add more:

Function Signature	Purpose
<code>void inputStudents(...)</code>	Reads and validates student data from the user
<code>void computeGrades(...)</code>	Calculates totals and assigns grades
<code>void displayReport(...)</code>	Prints the full class result table
<code>void displayStatistics(...)</code>	Prints highest, lowest, average, pass/fail counts
<code>int searchByID(...)</code>	Returns the index of a student with a given ID, or -1 if not found
<code>char assignGrade(float total)</code>	Returns the letter grade for a given total score

4.2 Mandatory Use of Pointers

You must use pointers in at least TWO of the following ways to demonstrate understanding:

- Pass arrays to functions using pointer notation (`int* arr`) instead of array notation.
- Use a pointer variable to traverse an array (pointer arithmetic).
- Pass scalar variables by pointer (e.g. to return multiple values from a function such as `findHighestLowest`).
- Use a pointer to swap two values in a helper swap function.

4.3 Code Quality Standards

- Your source file must begin with a comment block: your name, student ID, date, and a brief program description.
- Every function must have a comment explaining what it does, its parameters, and its return value.
- Use meaningful variable names (e.g. `totalScore`, not `ts`).
- Indent your code consistently — 4 spaces per level.
- Avoid global variables; pass all data through function parameters.

5. Deliverables

Submit the following by the deadline. Incomplete submissions will be penalised.

5.1 What to Submit

- `main.cpp` — a single, well-commented C++ source file containing your entire solution.
- `README.txt` — a plain-text file (10–15 lines) describing how to compile and run your program, and any known limitations.
- Project Report (2–3 pages, PDF or Word) — describing your design decisions, how you used arrays, functions, and pointers, and challenges you encountered.

5.2 Submission Instructions

- Name your ZIP file: `CS101_Project_[YourStudentID].zip`
- Submit via the course portal or hand a printed copy to the lab coordinator.
- Late submissions attract a deduction of 5 marks per day.

6. Hints & Common Pitfalls

Helpful Hints

- Plan before you code. Draw a flowchart or write pseudocode for each function first.
- Test each function independently using small data sets before putting everything together.
- Remember: an array name in C++ is already a pointer to its first element. So `int scores[30]` passed to a function is the same as `int* scores`.
- Use `setw()` and `setprecision()` from `<iomanip>` to align your output table neatly.
- For the search function, use a simple linear search loop — you are not required to implement a binary search.

Common Pitfalls to Avoid

- Off-by-one errors: if you have 5 students, valid indices are 0 through 4, not 0 through 5.
- Forgetting to initialise variables — always set `studentCount = 0` before the input loop.
- Confusing `=` (assignment) with `==` (comparison) inside if conditions.
- Leaving out the `&` (address-of) when you intend to pass by pointer.

Academic Integrity Warning

This is an INDIVIDUAL project. Collaboration to understand concepts is encouraged, but submitted code must be your own. Plagiarism or copying another student's work will result in a mark of zero for all students involved and will be reported to the department

7. Expected Sample Output

Your program's output does not need to look exactly like this, but it should be similarly clear and well-formatted.

```
=====
STUDENT GRADE MANAGEMENT SYSTEM
=====

1. Enter Student Records
2. Compute and Assign Grades
3. Display Full Class Report
4. Display Class Statistics
5. Search Student by ID
6. Exit
-----

Enter your choice: 3

      ID      NAME      TOTAL  GRADE
REMARK
-----
2024001  Ama Owusu      85.0    A    Pass
2024002  Kofi Mensah      72.5    B    Pass
2024003  Abena Asante      44.0    F    Fail

=====
CLASS STATISTICS
=====

Highest Score : 85.0 (Ama Owusu)
Lowest Score  : 44.0 (Abena Asante)
Class Average : 67.2
Students Passed: 2   Failed: 1
```

Good luck! Remember: a good programmer is not one who knows everything, but one who knows how to break a problem into smaller pieces and solve them one at a time.